

HDT5 Orquestación, respuestas

CC3116, Sistemas RAG. Parachute S.A., agente multiagente con OpenAI Agents SDK.

1. ¿Qué arquitectura o arquitecturas resuelven mejor este problema? ¿Por qué?

El problema tiene dos dominios de trabajo (FAQs del evento, y clima más agenda) y tres herramientas en total. Habiendo implementado y probado las tres arquitecturas contra el mismo problema, la respuesta corta es: la

centralizada es la que mejor encaja hoy, y la **jerárquica** es la que más sentido tiene pensando en el crecimiento que Parachute ya anunció. La **decentralizada** es la que peor encaja para este caso puntual, aunque el patrón de handoff sí tiene su lugar en otros problemas.

Centralizada. Un solo manager habla con el usuario y decide, para cada mensaje, a cuál especialista delegar (`as_tool``). Es la opción más simple de razonar, la más barata en llamadas al modelo (dos saltos: manager y especialista), y la que mejor resuelve preguntas que cruzan dominios en un mismo turno ("hay parqueo y de paso quiero agendar"), porque el manager puede llamar ambas tools y combinar la respuesta él mismo. Con solo dos especialistas y tres herramientas, no hay ninguna ventaja real en meter más niveles.

Jerárquica. Agrega un nivel extra (director sobre dos submanagers) sin cambiar el resultado para el usuario, solo agrega latencia (tres saltos en vez de dos, se notó claramente al probarla). Su valor no está en resolver el problema de hoy sino en anticipar el que viene: Parachute avisó que va a seguir agregando requisitos. Cuando la cantidad de herramientas crezca (más de cinco o seis), un solo manager plano empieza a fallar en elegir la tool correcta entre muchas opciones parecidas; agruparlas por dominio bajo submanagers mantiene la lista de opciones de cada agente chica y más fácil de acertar. Es la arquitectura más cara de correr hoy, pero la que mejor escala si el catálogo de funcionalidades sigue creciendo como Parachute advirtió.

Decentralizada. Es la que peor encaja acá. El patrón de handoff está pensado para tareas donde, una vez que un especialista toma la conversación, tiene sentido que se quede a cargo por varios turnos seguidos (por ejemplo, una escalación a soporte técnico). En este problema el usuario salta de tema constantemente (una pregunta de FAQ, después quiere agendar, después otra pregunta de FAQ), lo que obliga a los agentes a estar transfiriéndose la

conversación de un lado a otro todo el tiempo. Además, en la práctica esta fue la arquitectura más frágil de las tres: aparecieron dos problemas reales de compatibilidad entre el SDK y Groq (un schema de handoff que Groq rechaza, y el agente que recibe el turno confundiendo y contestando sin usar su propia herramienta) que no existen con `as\_tool`. Los dos se pudieron arreglar (detalle técnico en `hdt5/README.md`), pero el hecho de que hiciera falta arreglarlos ya dice algo: para un problema de dominios que se entrelazan todo el tiempo, `as\_tool` es un patrón más robusto que handoff.

2. ¿Considera que es necesario utilizar un sistema multiagente en este caso?

¿Por qué?

Para el alcance actual (buscar FAQs, consultar el clima, agendar una cita), no es estrictamente necesario: un solo agente con las tres herramientas registradas directamente habría resuelto el problema con una sola llamada al modelo por turno, más rápido y más barato que cualquiera de las tres arquitecturas multiagente, porque las tres tools no requieren personalidades, conocimientos o instrucciones tan distintas entre sí como para justificar agentes separados.

Lo que cambia la respuesta es que Parachute avisó explícitamente que va a seguir agregando requisitos funcionales. Ahí sí vale la pena pagar el costo extra de un sistema multiagente, no por el problema de hoy sino como inversión para el que sigue:

- Cada agente mantiene una lista corta de herramientas e instrucciones enfocadas en su dominio, en vez de un único prompt gigante con reglas para todo, que es donde los agentes (sobre todo los modelos más chicos, como se vio al probar `gpt-oss-20b`) empiezan a confundirse.
- Agregar una funcionalidad nueva es sumar una función en `shared/`, una tool, y conectarla al agente que corresponda, sin tocar el resto del sistema (por eso se armó la capa `shared/` de esta forma en HDT5).
- Da un límite natural para testear y iterar cada capacidad por separado.

En resumen: multiagente no es necesario por el tamaño del problema de hoy, pero sí es una decisión razonable dado que Parachute ya dijo que el sistema va a seguir creciendo, y la arquitectura elegida entre las tres (centralizada para hoy, con la puerta abierta a subir a jerárquica cuando la lista de herramientas crezca) es la que mejor balancea ese presente y ese futuro.